```
Post.all().length
# 3
```

If this isn't the behavior you want, then you'll need to wrap the export in a function in order to create a new scope every time it's `required`:

```
makePost = ->
  class Post
    posts = []
    constructor: (@title, @body) ->
      posts.push @
    @all: -> posts

  {Post}

exports.makePost = makePost
```

This `makePost` function now returns a different `Post` class each time it's evaluated:

```
{Post} = require('./post').makePost()
new Post 'A post', 'Some content'

Other = require ('./post').makePost().Post

Post.all().length
# 1

Other.all().length
# 0
```

In summary, be mindful that modules are loaded only once from source and then cached. This has implications for testing because often when running tests you'll want a clean state before each test. If you have a module that maintains some internal state, then you might find it difficult to test. It's best not to keep state in a module.

That's all there is to requiring local modules. Prefix the path to the module with either `./` or `../` and then specify the location of the file containing the module. A `require` that doesn't prefix the path with either of those will cause the module loader to look for a built-in or packaged module.[1]

[1] Creating built-in and packaged modules is outside the scope of this book.